Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_10_)

Implementation of Data Path

Student Name: ___Areeb Fatima_____ Roll No: _____24I-6543_____ Section: _B__

Date performed: ____28th October_____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this task, we have implemented the 32-bit complete data path and use all the modules as PC and PC+4 and Instruction Memory for data and then Register file for reading the source registers and then ALU for addi, add, sub, etc. operations given and then we use Datapath top module for connecting all the above modules and we don't implement the Data Memory in this Datapath as not required and simulate the testbench for verification.

Code:

```
// 1. PROGRAM COUNTER

module pc_reg(

   input clk,

   input reset,

   input [31:0] next_pc,

   output reg [31:0] pc

);

   always @(posedge clk or posedge reset) begin

      if (reset)

         pc <= 32'h00000000;

      else

         pc <= next_pc;

   end

endmodule

// 2. PC + 4 ADDER

module pc_adder(

   input [31:0] pc,

   output [31:0] pc4
```

```
);

   assign pc4 = pc + 32'd4;

endmodule

// 3. PC SELECTION MUX

module pc_mux(

   input [31:0] pc4,

   input [31:0] target,

   input PCSel,

   output [31:0] next_pc

);

   assign next_pc = PCSel ? target : pc4;

endmodule

// 4. INSTRUCTION MEMORY

module instruction_memory(

   input [31:0] pc,

   output reg [31:0] inst

);

   reg [31:0] mem [0:15];

   always @(*) begin

      if (pc[31:6] == 0)

         inst = mem[pc[5:2]];

      else

         inst = 32'h00000013; // NOP
```

```
    end

endmodule

// 5. REGISTER FILE

module register_file(

    input clk,

    input RegWEn,

    input [4:0] AddrA,

    input [4:0] AddrB,

    input [4:0] AddrD,

    input [31:0] DataD,

    output [31:0] DataA,

    output [31:0] DataB

);

    reg [31:0] registers [0:31];

    integer i;

    initial begin

        for (i = 0; i < 32; i = i + 1)

            registers[i] = 32'd0;

    end

    assign DataA = (AddrA == 5'd0) ? 32'd0 : registers[AddrA];

    assign DataB = (AddrB == 5'd0) ? 32'd0 : registers[AddrB];

    always @(posedge clk) begin

        if (RegWEn &&   (AddrD != 5'd0))
```

```
        registers[AddrD] <= DataD;
    end
endmodule
// 6. IMMEDIATE GENERATOR
module immediate_generator(
    input [31:0] inst,
    input [1:0] ImmSel,
    output reg [31:0] Imm
);
    always @(*) begin
        case (ImmSel)
            // I-TYPE
            2'b00:
                Imm = {{20{inst[31]}}, inst[31:20]};
            // S-TYPE
            2'b01:
                Imm = {{20{inst[31]}},
                    inst[31:25],
                    inst[11:7]};
            // B-TYPE
            2'b10:
                Imm = {{19{inst[31]}},
                    inst[31],
```

```
                inst[7],
                inst[30:25],
                inst[11:8],
                1'b0};
        // J-TYPE
        2'b11:
           Imm = {{12{inst[31]}},
                inst[19:12],
                inst[20],
                inst[30:21],
                1'b0};
        default:
           Imm = 32'd0;
      endcase
   end
endmodule
// 7. BRANCH COMPARATOR
module branch_comparator(
   input [31:0] DataA,
   input [31:0] DataB,
   output BrEq,
   output BrLT
);
```

```
    assign BrEq = (DataA == DataB);

    assign BrLT = ($signed(DataA) < $signed(DataB));

endmodule

// 8. ALU

module alu(

    input [31:0] A,

    input [31:0] B,

    input [2:0] ALUSel,

    output reg [31:0] result

);

    always @(*) begin

        case (ALUSel)

            3'b000: result = A + B; // ADD

            3'b001: result = A - B; // SUB

            3'b010: result = A & B; // AND

            3'b011: result = A | B; // OR

            3'b100: result = A ^ B; // XOR

            3'b101:

                result = ($signed(A) < $signed(B))

                        ? 32'd1 : 32'd0;

            default:

                result = 32'd0;

        endcase
```

```
    end

endmodule

// 9. ALU OPERAND MUXES

module alu_muxes(

    input [31:0] DataA,

    input [31:0] DataB,

    input [31:0] pc,

    input [31:0] Imm,

    input ASel,

    input BSel,

    output [31:0] A,

    output [31:0] B

);

    assign A = ASel ? pc : DataA;

    assign B = BSel ? Imm : DataB;

endmodule

// 10. WRITE BACK MUX

module writeback_mux(

    input [31:0] ALUResult,

    input [31:0] pc4,

    input [31:0] MemData,

    input [1:0] WBSel,

    output reg [31:0] wb
```

```
);

   always @(*) begin

      case (WBSel)

         2'b00: wb = ALUResult;

         2'b01: wb = pc4;

         2'b10: wb = MemData;

         default: wb = 32'd0;

      endcase

   end

endmodule

// 11. COMPLETE DATAPATH

module datapath_top(

   input clk,

   input reset,

   input PCSel,

   input RegWEn,

   input ASel,

   input BSel,

   input [2:0] ALUSel,

   input [1:0] ImmSel,

   input [1:0] WBSel,

   output [31:0] pc,

   output [31:0] inst,
```

```
    output [31:0] alu_result,
    output [31:0] wb,
    output BrEq,
    output BrLT,
    // Debug signals
    output [31:0] DataA,
    output [31:0] DataB,
    output [31:0] Imm,
    output [31:0] pc4,
    output [31:0] target
);
    wire [31:0] next_pc;
    wire [31:0] A;
    wire [31:0] B;
    wire [31:0] MemData;
    assign MemData = 32'd0;
    // Program Counter
    pc_reg PC(
        .clk(clk),
        .reset(reset),
        .next_pc(next_pc),
        .pc(pc)
    );
```

```
// PC + 4
pc_adder PC4(
   .pc(pc),
   .pc4(pc4)
);
// Immediate generator
immediate_generator IMM(
   .inst(inst),
   .ImmSel(ImmSel),
   .Imm(Imm)
);
// Instruction memory
instruction_memory IMEM(
   .pc(pc),
   .inst(inst)
);
// Branch target
assign target = pc + Imm;
// PC selection
pc_mux PCMUX(
   .pc4(pc4),
   .target(target),
   .PCSel(PCSel),
```

```
    .next_pc(next_pc)
);
// Register file
register_file RF(
    .clk(clk),
    .RegWEn(RegWEn),
    .AddrA(inst[19:15]),
    .AddrB(inst[24:20]),
    .AddrD(inst[11:7]),
    .DataD(wb),
    .DataA(DataA),
    .DataB(DataB)
);
// Branch comparator
branch_comparator BC(
    .DataA(DataA),
    .DataB(DataB),
    .BrEq(BrEq),
    .BrLT(BrLT)
);
// ALU input selection
alu_muxes AMUX(
    .DataA(DataA),
```

```
    .DataB(DataB),
    .pc(pc),
    .Imm(Imm),
    .ASel(ASel),
    .BSel(BSel),
    .A(A),
    .B(B)
);
// ALU
alu ALU(
    .A(A),
    .B(B),
    .ALUSel(ALUSel),
    .result(alu_result)
);
// Write back
writeback_mux WBMUX(
    .ALUResult(alu_result),
    .pc4(pc4),
    .MemData(MemData),
    .WBSel(WBSel),
    .wb(wb)
);
```

```
endmodule

module tb_datapath();

   // Inputs

   reg clk;

   reg reset;

   reg PCSel;

   reg RegWEn;

   reg ASel;

   reg BSel;

   reg [2:0] ALUSel;

   reg [1:0] ImmSel;

   reg [1:0] WBSel;

   // Outputs

   wire [31:0] pc;

   wire [31:0] inst;

   wire [31:0] alu_result;

   wire [31:0] wb;

   wire BrEq;

   wire BrLT;

   // Debug signals

   wire [31:0] DataA;

   wire [31:0] DataB;

   wire [31:0] Imm;
```

```
wire [31:0] pc4;

wire [31:0] target;

// Instantiate datapath

datapath_top uut (

   .clk(clk),

   .reset(reset),

   .PCSel(PCSel),

   .RegWEn(RegWEn),

   .ASel(ASel),

   .BSel(BSel),

   .ALUSel(ALUSel),

   .ImmSel(ImmSel),

   .WBSel(WBSel),

   .pc(pc),

   .inst(inst),

   .alu_result(alu_result),

   .wb(wb),

   .BrEq(BrEq),

   .BrLT(BrLT),

   .DataA(DataA),

   .DataB(DataB),

   .Imm(Imm),

   .pc4(pc4),
```

```
    .target(target)
);
// Clock: 10 ns period
always #5 clk = ~clk;
initial begin
    // Initialize inputs
    clk = 0;
    reset = 1;
    PCSel = 0;
    RegWEn = 0;
    ASel = 0;
    BSel = 0;
    ALUSel = 3'b000;
    ImmSel = 2'b00;
    WBSel = 2'b00;
    // Load six instructions
    uut.IMEM.mem[0] = 32'h00700293; // ADDI x5, x0, 7
    uut.IMEM.mem[1] = 32'h00500313; // ADDI x6, x0, 5
    uut.IMEM.mem[2] = 32'h006283b3; // ADD  x7, x5, x6
    uut.IMEM.mem[3] = 32'h40628433; // SUB  x8, x5, x6
    uut.IMEM.mem[4] = 32'h00038063; // BEQ  x7, x0, offset
    uut.IMEM.mem[5] = 32'h008384b3; // ADD  x9, x7, x8
    // Initialize remaining memory with NOPs
```

```
uut.IMEM.mem[6]  = 32'h00000013;

uut.IMEM.mem[7]  = 32'h00000013;

uut.IMEM.mem[8]  = 32'h00000013;

uut.IMEM.mem[9]  = 32'h00000013;

uut.IMEM.mem[10] = 32'h00000013;

uut.IMEM.mem[11] = 32'h00000013;

uut.IMEM.mem[12] = 32'h00000013;

uut.IMEM.mem[13] = 32'h00000013;

uut.IMEM.mem[14] = 32'h00000013;

uut.IMEM.mem[15] = 32'h00000013;

// Keep reset active

#10;

reset = 0;

// Instruction 1: ADDI x5, x0, 7

// x5 = 0 + 7 = 7

PCSel = 0;

RegWEn = 1;

ASel = 0;

BSel = 1;

ALUSel = 3'b000;

ImmSel = 2'b00;

WBSel = 2'b00;

#10;
```

```
// Instruction 2: ADDI x6, x0, 5

// x6 = 0 + 5 = 5

PCSel = 0;

RegWEn = 1;

ASel = 0;

BSel = 1;

ALUSel = 3'b000;

ImmSel = 2'b00;

WBSel = 2'b00;

#10;

// Instruction 3: ADD x7, x5, x6

// x7 = 7 + 5 = 12

PCSel = 0;

RegWEn = 1;

ASel = 0;

BSel = 0;

ALUSel = 3'b000;

WBSel = 2'b00;

#10;

// Instruction 4: SUB x8, x5, x6

// x8 = 7 - 5 = 2

PCSel = 0;

RegWEn = 1;
```

```
ASel = 0;

BSel = 0;

ALUSel = 3'b001;

WBSel = 2'b00;

#10;

// Instruction 5: BEQ x7, x0, offset

// x7 = 12, x0 = 0

// BrEq = 0, branch not taken

PCSel = 0;

RegWEn = 0;

ASel = 0;

BSel = 0;

ImmSel = 2'b10;

#10;

// Instruction 6: ADD x9, x7, x8

// x9 = 12 + 2 = 14

PCSel = 0;

RegWEn = 1;

ASel = 0;

BSel = 0;

ALUSel = 3'b000;

WBSel = 2'b00;

#10;
```

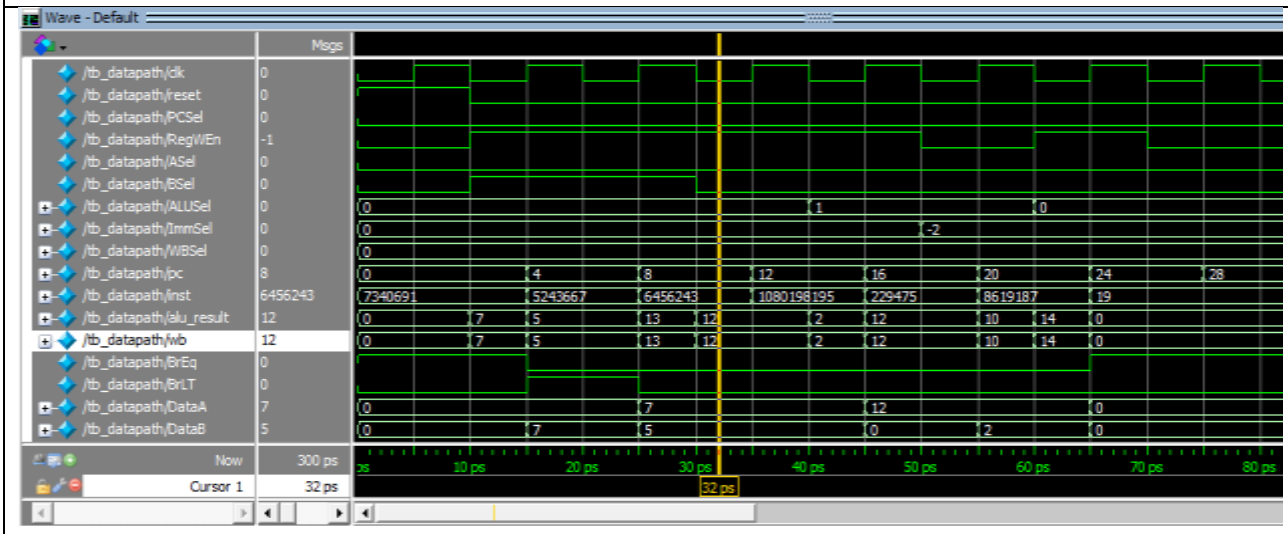
```
    // Stop simulation

    RegWEn = 0;

  end

endmodule
```

Output:



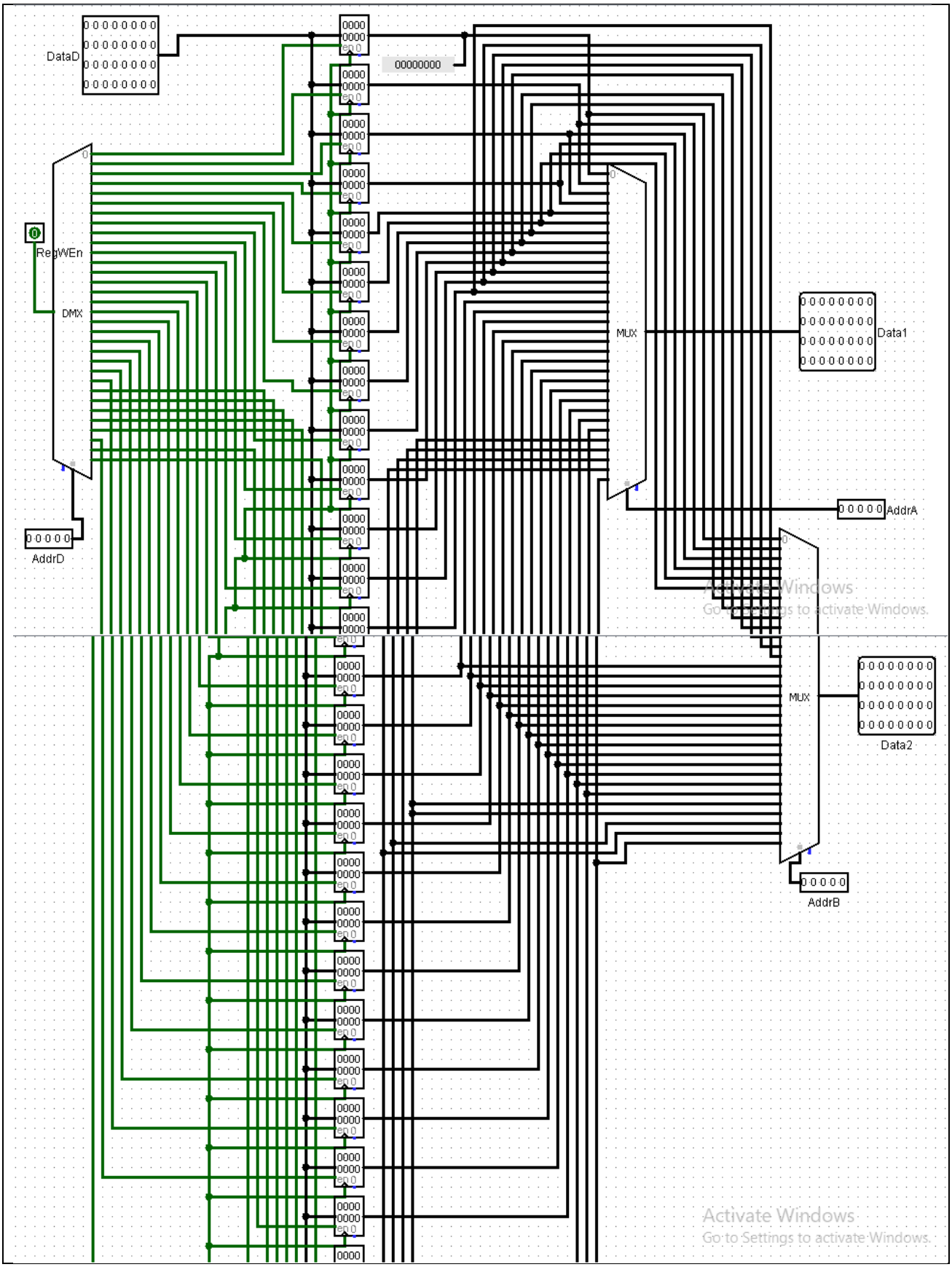
Task 2:

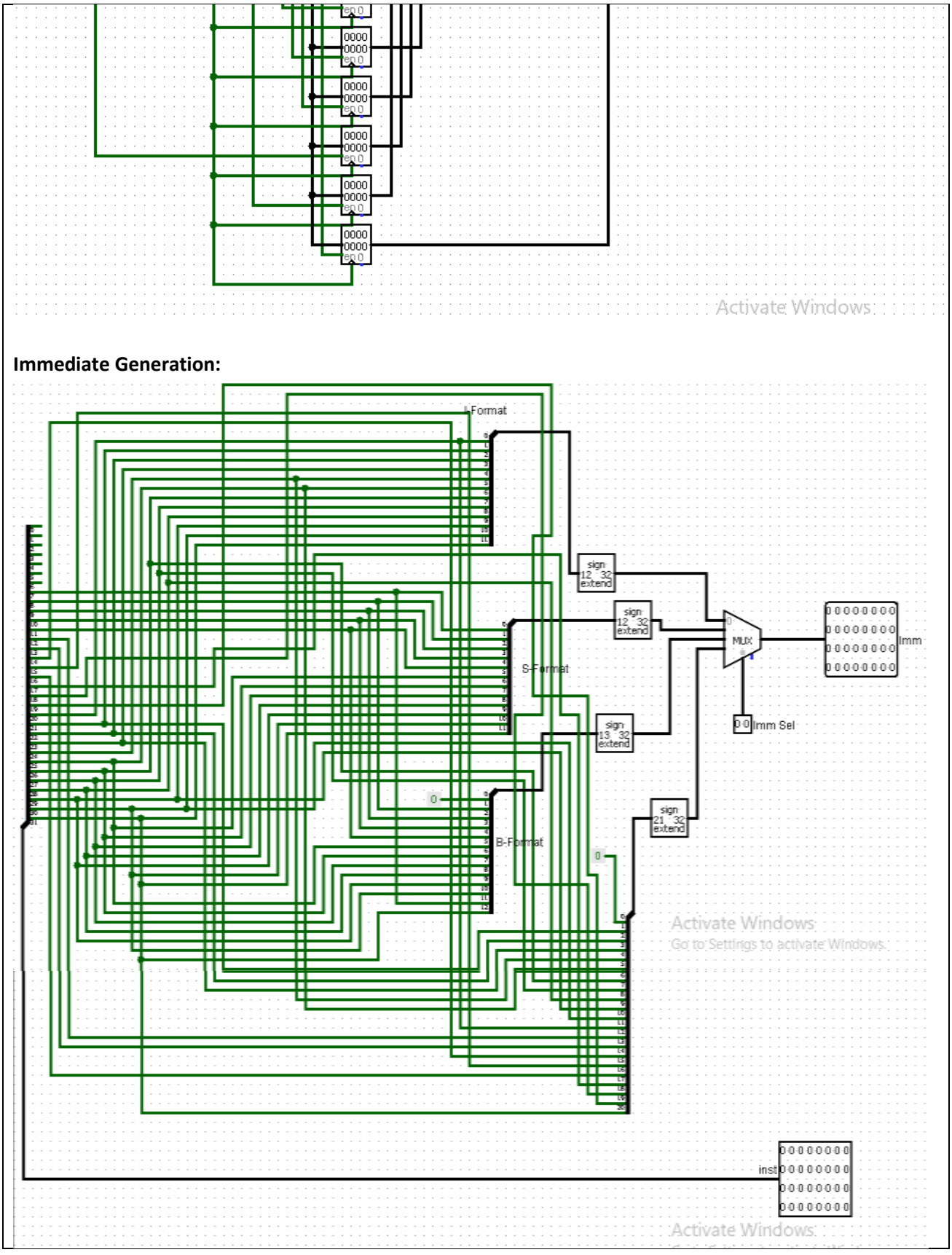
Description:

In this task, we made the separate modules for Instruction Memory and ALU and Register file and separately ran them for better understanding and after their successful run, we made the better understanding for Datapath execution in Logisim accordingly by making separate sub circuits for every module.

Output:

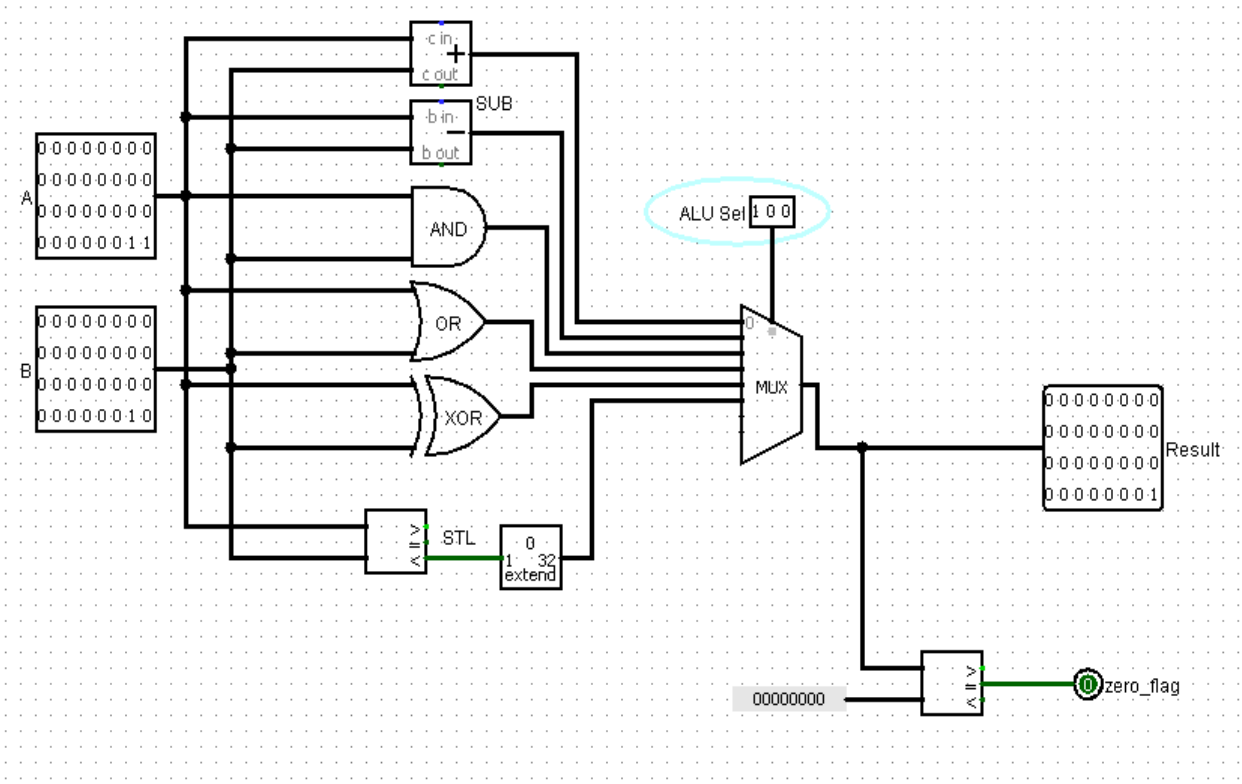
Register File:


DataD
00000000
RegWEn
DMX
AddrD
MUX
Data1
AddrA
MUX
Data2
AddrB
Activate Windows
Go to Settings to activate Windows.

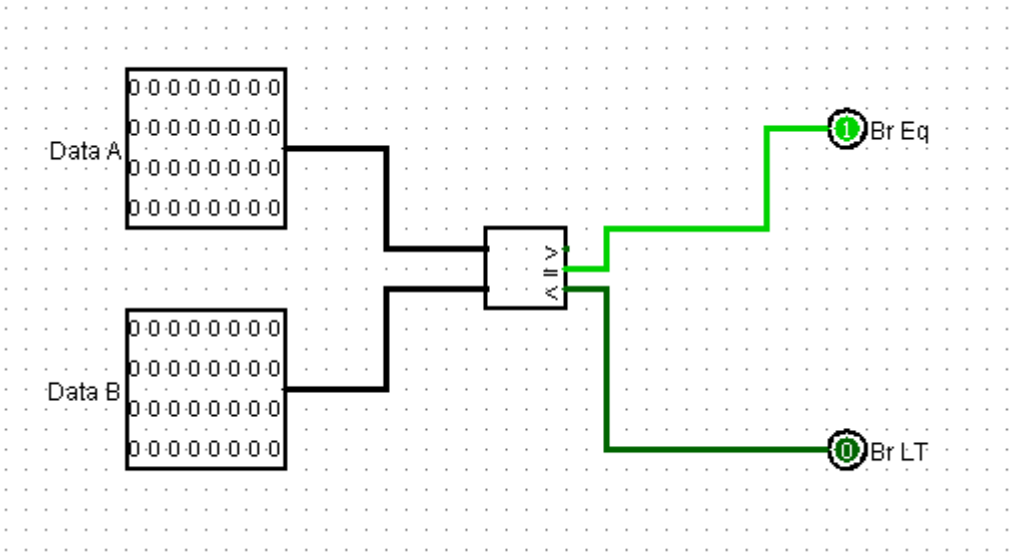


Immediate Generation:

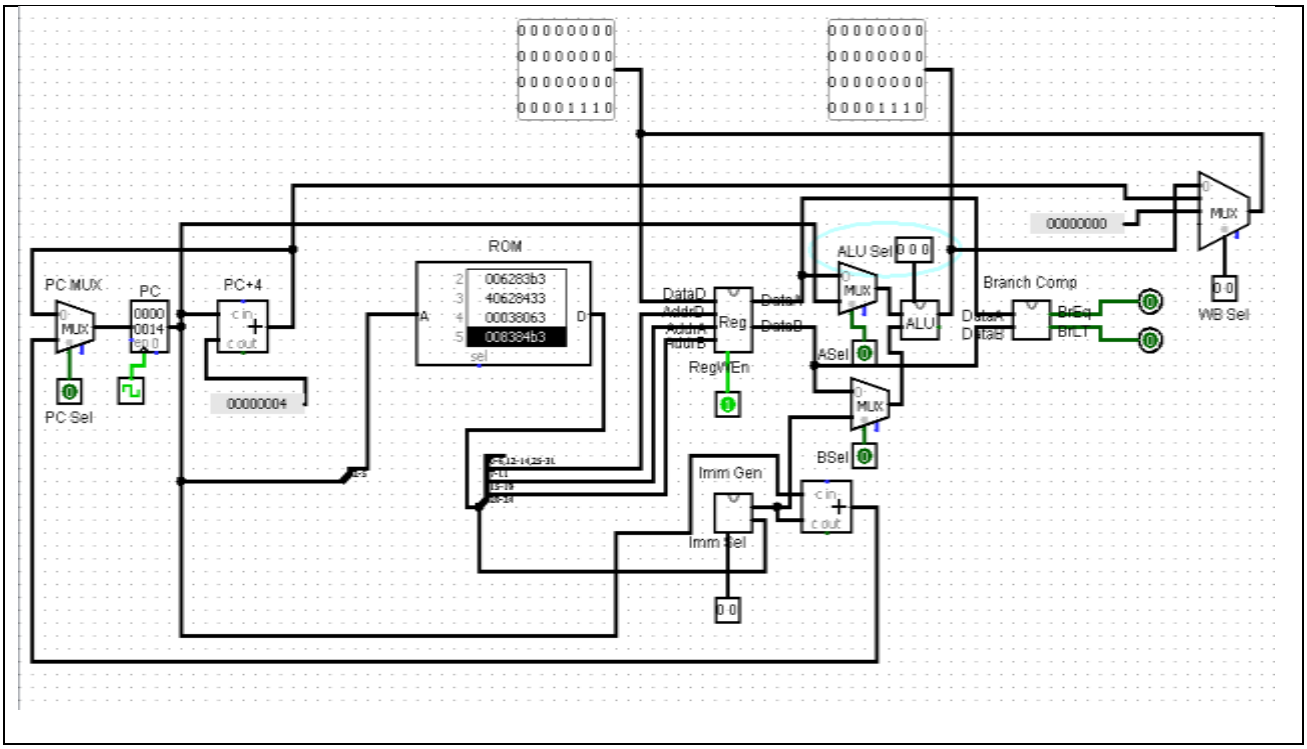
ALU:



Branch Comparator:



Data Path:


ROM
006283b3
40628433
00038063
008384b3
PC MUX
PC
PC+4
PC Sel
00000004
DataD
Reg
RegWEn
DataA
DataB
ALU Sel
ALU
ASel
BSel
Branch Comp
BrEq
BrLT
Imm Gen
Imm Sel
00000000
WB Sel